



VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMACINIŲ SISTEMŲ INŽINERIJOS STUDIJŲ PROGRAMA

Tiesinis programavimas

4 Laboratorinio darbo ataskaita

Atliko: Adomas Lukoševičius

VU el. p.: adomas.lukosevicius@mif.stud.vu.lt

Vertino: prof. dr. Julius Žilinskas

UŽDUOTIS

1. SUPROGRAMUOKITE SIMPLEKSO ALGORITMĄ TIESINIO PROGRAMAVIMO UŽDAVINIAMS

Naudojant *Golang* programavimo kalbą, implementavau simplekso algoritmą tiesinio programavimo uždaviniams. Algoritmo kodas įkeltas priede ataskaitos gale.

2. UŽRAŠYKITE DUOTĄ UŽDAVINĮ MATRICINIŲ PAVIDALU STANDARTINE FORMA.

Buvo duotas tiesinio programavimo uždavinys:

$$\min f = 2x_1 - 3x_2 - 5x_4$$

Su apribojimais:

$$-x_1 + x_2 - x_3 - x_4 \leq 8$$

$$2x_1 + 4x_2 \leq 10$$

$$x_3 + x_4 \leq 3$$

$$x_i \geq 0$$

Kadangi Simplekso algoritmas ieško funkcijos maksimumo, tikslo funkciją f perrašome pagal $\max g = -f$:

$$\max g = -2x_1 + 3x_2 + 5x_4$$

Uždutis buvo perrašyta matriciniu pavidalu standartine forma $\min c^T x$, kai $Ax \leq b$, kur:

$$A = \begin{bmatrix} -1 & 1 & -1 & -1 \\ 2 & 4 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

$$b = \begin{bmatrix} 8 \\ 10 \\ 3 \end{bmatrix}$$

Iš lygties $g = 2x_1 - 3x_2 - 5x_4$ gaunamas koeficientų vektorius:

$$c = \begin{bmatrix} -2 \\ 3 \\ 0 \\ 5 \end{bmatrix}$$

Galiausiai, pridėdame s_i kintamuosius, kad apribojimų nelygybes paverstume lygybėmis

$$A = \begin{bmatrix} -1 & 1 & -1 & -1 & 1 & 0 & 0 \\ 2 & 4 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

3. IŠSPRĘSKITE UŽDAVINĮ SUPROGRAMUOTU SIMPLEKSO ALGORITMU.

Naudojant anksčiau gautas A , b ir c matricas, buvo paleistas suprogramuotas simplekso algoritmas.

Simplekso algoritmas buvo sprendžiamas per kelias iteracijas, *tableau* principu. Žemiau pateikiama kiekviena uždavinio iteracija:

iteracija 0									
	x1	x2	x3	x4	s1	s2	s3	desine	puse
s1	-1	1	-1	-1	1	0	0	8	
s2	2	4	0	0	0	1	0	10	
s3	0	0	1	1	0	0	1	3	
z	2	-3	-0	-5	-0	-0	-0	0	

pav. 1 Pradiniai duomenys

iteracija 1									
	x1	x2	x3	x4	s1	s2	s3	desine	puse
s1	-1	1	0	0	1	0	1	11	
s2	2	4	0	0	0	1	0	10	
x4	0	0	1	1	0	0	1	3	
z	2	-3	5	0	0	0	5	15	

pav. 2 Duomenys po 1 iteracijos

iteracija 2									
	x1	x2	x3	x4	s1	s2	s3	desine	puse
s1	-1.5	0	0	0	1	-0.25	1	8.5	
x2	0.5	1	0	0	0	0.25	0	2.5	
x4	0	0	1	1	0	0	1	3	
z	3.5	0	5	0	0	0.75	5	22.5	

pav. 3 Duomenys po 2 iteracijos

```

----- rezultatas -----
z_max = 22.5
f_min = -22.5
x* = [x1=0, x2=2.5, x3=0, x4=3]
galutine baze = [s1 x2 x4]

```

pav. 4 Galutiniai duomenys

4. PAKEISKITE APRIBOJIMŲ DEŠINĖS PUSĖS KONSTANTAS Į A, B IR C – STUDENTO KNYGELĖS NUMERIO “2X1XABC” SKAITMENIS. IŠSPRĘSKITE INDIVIDUALŲ UŽDAVINĮ SUPROGRAMUOTU SIMPLEKSO ALGORITMU.

Pagal studento kodą, nustačiau A, B ir C reikšmes 9, 3 ir 9 (žr. pav. 5).

```

// mano studento skaitmenys
var A float64 = 9
var B float64 = 3
var C float64 = 9

```

pav. 5 mano studento skaitmenys

Šias reikšmes įsistačius į pradinę B matricą, gauname naują matricą:

$$b = \begin{bmatrix} 9 \\ 3 \\ 9 \end{bmatrix}$$

Įstatę naujai gautus kintamuosius į algoritmą, gauname tokią išvestį:

```

iteracija 0
  x1  x2  x3  x4  s1  s2  s3  desine  puse
s1 -1  1  -1  -1  1  0  0  9
s2  2  4  0  0  0  1  0  3
s3  0  0  1  1  0  0  1  9
z   2  -3  -0  -5  -0  -0  -0  0

```

pav. 6 Pradiniai duomenys

```

iteracija 1
  x1  x2  x3  x4  s1  s2  s3  desine  puse
s1 -1  1  0  0  1  0  1  18
s2  2  4  0  0  0  1  0  3
x4  0  0  1  1  0  0  1  9
z   2  -3  5  0  0  0  5  45

```

pav. 7 Duomenys po 1 iteracijos

iteracija 2									
	x1	x2	x3	x4	s1	s2	s3	desine	puse
s1	-1.5	0	0	0	1	-0.25	1	17.25	
x2	0.5	1	0	0	0	0.25	0	0.75	
x4	0	0	1	1	0	0	1	9	
z	3.5	0	5	0	0	0.75	5	47.25	

pav. 8 Duomenys po 2 iteracijos

```

----- rezultatas -----
z_max = 47.25
f_min = -47.25
x* = [x1=0, x2=0.75, x3=0, x4=9]
galutine baza = [s1 x2 x4]

```

pav. 9 Galutiniai duomenys

5. PALYGINKITE UŽDAVINIŲ SPRENDIMO REZULTATUS.

Žemiau pateiktas pirmo ir antro uždavinio rezultatų tyrimas

```

===== palyginimas =====
Pirmas uždavinys: f_min=-22.5, x=[0 2.5 0 3], baza=[s1 x2 x4]
Antras uždavinys: f_min=-47.25, x=[0 0.75 0 9], baza=[s1 x2 x4]

```

pav. 10 1 ir 2 uždavinių palyginimas

Matoma, kad minimali funkcijos reikšmė (f_{min}) pirmame uždavinyje lygi -22,5, o antrame -47,25. Matoma, kad apribojimo funkcijos gali turėti didelę (šiuo atveju daugiau negu 2x) įtaką funkcijos minimumo reikšmei. Be to matoma, kad abiejų uždavinių atveju gauta ta pati bazė.

ALGORITMO KODAS

```
func simplex(A [][]float64, b []float64, c []float64, basis0 []string, hook
func(iter int, tableau [][]float64, basis []string)) (zMax float64, x []float64,
basis []string, tableau [][]float64) {
    const eps = 1e-9
    m := len(A)
    n := len(A[0])
    basis = append([]string(nil), basis0...)

    // sukuriame pradini tableau
    // m - apribojimu + 1 uždavinio eilutes
    // n - kintamųjų + desines puses stulpeliai
    tableau = make([][]float64, m+1)
    for i := 0; i < m; i++ {
        row := make([]float64, n+1)
        copy(row, A[i])
        row[n] = b[i]
        tableau[i] = row
    }
    obj := make([]float64, n+1)
    for j := 0; j < n; j++ {
        obj[j] = -c[j]
    }
    tableau[m] = obj

    if hook != nil {
        hook(0, tableau, basis)
    }

    for iter := 1; ; iter++ {
        enter := -1
        best := -eps
        for j := 0; j < n; j++ {
            if tableau[m][j] < best {
                best = tableau[m][j]
                enter = j
            }
        }
        if enter == -1 {
            break
        }

        leave := -1
        minRatio := 0.0
        for i := 0; i < m; i++ {
            a := tableau[i][enter]
            if a > eps {
                r := tableau[i][n] / a
                if leave == -1 || r < minRatio-eps {
                    minRatio = r
                }
            }
        }
    }
}
```

```

        leave = i
    }
}
pivot := tableau[leave][enter]

for j := 0; j <= n; j++ {
    tableau[leave][j] /= pivot
}
for i := 0; i <= m; i++ {
    if i == leave {
        continue
    }
    factor := tableau[i][enter]
    if math.Abs(factor) < eps {
        continue
    }
    for j := 0; j <= n; j++ {
        tableau[i][j] -= factor * tableau[leave][j]
    }
}

basis[leave] = kintamieji[enter]
if hook != nil {
    hook(iter, tableau, basis)
}
}

// istraukti x1..x4.
xAll := make([]float64, n)
for i := 0; i < m; i++ {
    col := -1
    for j, name := range kintamieji {
        if name == basis[i] {
            col = j
            break
        }
    }
    if col >= 0 {
        xAll[col] = tableau[i][n]
    }
}

zMax = tableau[m][n]
x = append([]float64(nil), xAll[:4]...)
return zMax, x, basis, tableau
}

```